

Q.1 Maximise Subarray Sum

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> maxSubarraySum (int N, int K, vector<int> A) {
    // min-heap to store the chosen elements (at most K largest positives)
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    long long curSum = 0;
    vector<long long> ans(N);

    for (int i = 0; i < N; ++i) {
        long long x = A[i];

        if (x > 0) {
            if ((int)pq.size() < K) {
                // still room: just add this element
                pq.push(x);
                curSum += x;
            } else if (pq.top() < x) {
                // replace smallest chosen element with this bigger one
                curSum -= pq.top();
                pq.pop();
                pq.push(x);
                curSum += x;
            }
        }
        // best sum using at most K elements from A[0..i]
        ans[i] = curSum;
    }

    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int N, K;
        cin >> N >> K;
        vector<int> A(N);
        for (int i = 0; i < N; ++i) cin >> A[i];

        vector<long long> res = maxSubarraySum(N, K, A);
        for (int i = 0; i < N; ++i) {
```

```

        if (i) cout << ' ';
        cout << res[i];
    }
    cout << "\n";
}
return 0;
}

```

Q.2 Tricky Subarray

```

#include <bits/stdc++.h>
using namespace std;

```

```

vector<long long> TrickySubarrays(int N, vector<int> A, int Q, vector<vector<int>> Queries) {
    vector<long long> res;

```

```

    // shift to 1-based
    vector<int> arr(N + 1);
    for (int i = 1; i <= N; i++) arr[i] = A[i - 1];

```

```

    vector<int> pos(N + 1);
    for (int i = 1; i <= N; i++) pos[arr[i]] = i;

```

```

    vector<int> prefix_min(N + 1), prefix_max(N + 1);
    int mn = INT_MAX, mx = 0;

```

```

    for (int k = 1; k <= N; k++) {
        mn = min(mn, pos[k]);
        mx = max(mx, pos[k]);
        prefix_min[k] = mn;
        prefix_max[k] = mx;
    }

```

```

    for (auto &q : Queries) {
        int L = q[0], R = q[1];

```

```

        int low = 1, high = N, ans = 0;
        while (low <= high) {

```

```

        int mid = (low + high) >> 1;
        if (prefix_min[mid] >= L && prefix_max[mid] <= R) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    res.push_back(ans);
}

return res;
}

```

Q.3 Trim Zeroes

```

#include <bits/stdc++.h>
using namespace std;

vector<int> solve(int N, string S, int Q, vector<vector<int>> &Queries) {
    vector<int> res;
    for (auto &q : Queries) {
        if (q[0] == 1) {
            int D = q[1], X = q[2];
            S[D - 1] = (X == 1 ? '1' : '0');
        } else {
            int l = q[1], r = q[2];
            string A = S.substr(l - 1, r - l + 1);
            size_t first = A.find('1');
            if (first == string::npos) {
                res.push_back(0);
            } else {
                size_t last = A.rfind('1');
                res.push_back(last - first + 1);
            }
        }
    }
    return res;
}

```

Q.4 Sum of all nodes Binary tree

```

#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<long long> A, subtree, pathsum;

long long dfs_subtree(int node, int parent) {
    subtree[node] = A[node];
    for (int child : adj[node])
        if (child != parent)
            subtree[node] += dfs_subtree(child, node);
    return subtree[node];
}

void dfs_pathsum(int node, int parent, long long sum) {
    pathsum[node] = sum + A[node];
    for (int child : adj[node])
        if (child != parent)
            dfs_pathsum(child, node, pathsum[node]);
}

long long solve(int N, vector<int> &arr, vector<vector<int>> &C) {
    A.assign(N + 1, 0);
    subtree.assign(N + 1, 0);
    pathsum.assign(N + 1, 0);
    adj.assign(N + 1, {});

    for (int i = 1; i <= N; i++) A[i] = arr[i - 1];

    for (int i = 0; i < N; i++) {
        int L = C[i][0], R = C[i][1];
        if (L != -1) {
            adj[i + 1].push_back(L);
            adj[L].push_back(i + 1);
        }
        if (R != -1) {
            adj[i + 1].push_back(R);
            adj[R].push_back(i + 1);
        }
    }

    dfs_subtree(1, -1);
    dfs_pathsum(1, -1, 0);

    long long ans = 0;
    for (int v = 1; v <= N; v++) {
        long long S1 = subtree[v] - A[v];
    }
}

```

```

        long long S2 = pathsum[v];
        ans += labs(S1 - S2);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    for (int t = 0; t < T; t++) {
        int N;
        cin >> N;
        vector<int> A(N);
        for (int i = 0; i < N; i++) cin >> A[i];
        vector<vector<int>> C(N, vector<int>(2));
        for (int i = 0; i < N; i++)
            for (int j = 0; j < 2; j++)
                cin >> C[i][j];
        cout << solve(N, A, C) << "\n";
    }
    return 0;
}

```